

Personalized Tourist Itinerary Recommendation

The Smart Visit Module — the whole thesis, explained

*From a personalized, context-aware next-POI engine to literature-comparable
itineraries — testing the field's integration hypothesis*

Companion document to the defence deck

Repository: github.com/6ym6n/PFE_IMPLEMENTATION

Executive summary — the whole idea

This thesis builds a **Smart Visit Module** that recommends **personalized, context-aware tourist itineraries** — *ordered routes* of places to visit, tailored to the user. Its core is a **next-POI engine**: a graph neural network + sequence model that uses **who the user is** (a user embedding — the title's *user preferences*) and **context** (distance Δd and time Δt between places — the title's *contextual data*) to score the next place to visit.

The thesis then confronts the field's central question — raised by Halder's 2024 survey and the DLIR model (2025): should **choosing good POIs** and **ordering them into a route** be **separate** steps, or **integrated**? It builds **both** and compares them: **Strategy A** decodes the frozen engine into a route (decoupled), and **Strategy B** trains a dedicated itinerary model end-to-end (integrated). The **honest finding**: on Foursquare NYC the integrated model did *not* beat the decoupled one — because the engine was trained on far more examples. Integration is not automatically better; **supervision density matters**.

Finally, because the NYC itinerary scores are **not comparable** to the published literature (a different benchmark), the thesis re-runs everything on the standard **Flickr** datasets under the **exact published protocol (Strategy D)**. There a **Random baseline reproduces Chen 2016** (proving the setup is fair), and the methods land on the published **0.3–0.85 pairs-F1 scale**. The contribution: a personalized, context-aware engine + an honest decoupled-vs-integrated study + a validated literature comparison — testing, and nuancing, the field's integration hypothesis, with a clear path to a full scheduling-aware module.

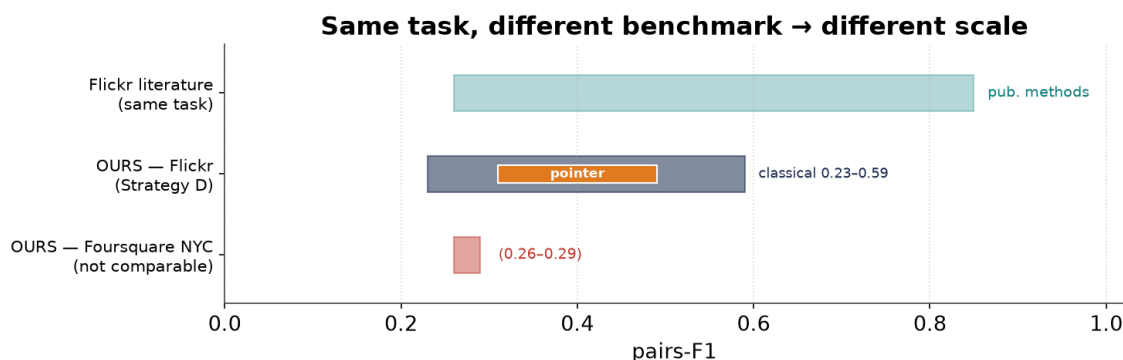


Figure 1 — Why Strategy D exists: the same task on a different benchmark sits on a different scale. Our Flickr work puts us on the literature's scale.

1. The goal — a Smart Visit Module

A tourist with limited time faces hundreds of places and wants a **personalized day plan**: an ordered route of POIs (Points Of Interest — places worth visiting). A good plan depends on the **user** (their preferences) and the **context** (where they are, how far apart places are, the time). This is **itinerary recommendation** — producing the whole ordered route — which is a different problem from **next-POI prediction** (guessing only the single next place).

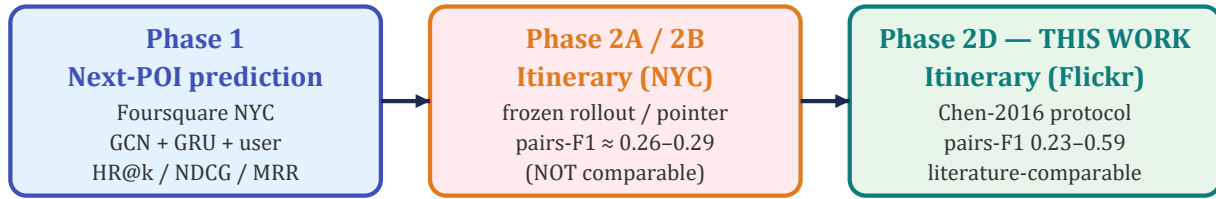


Figure 2 — Where the pieces sit. Phase 1 (next-POI) is the engine; Phase 2 turns it into itineraries on Foursquare NYC; Strategy D validates on the Flickr benchmark.

2. Background — places, trips, queries, and the score

A **trajectory** (a trip) is the ordered list of POIs one tourist visited. An itinerary **query** gives the **start POI**, the **end POI**, and the **length K**; the model must recover the ordered POIs in between.

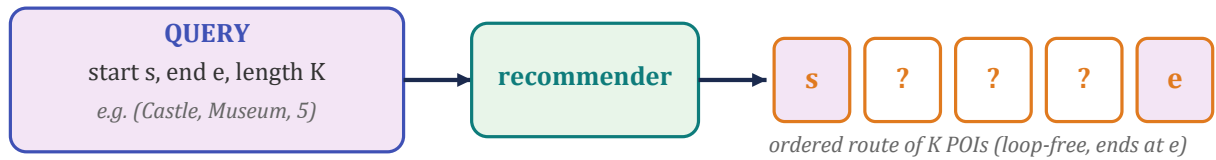
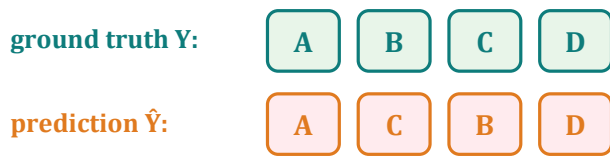


Figure 3 — The query gives the endpoints and the length; the model fills the ordered middle.

Quality is measured by two scores in $[0, 1]$. **point-F1** asks *did we pick the right places?* (ignoring order). **pairs-F1** (Chen 2016, the main metric) asks *did we get the order right too?* — it is the F1 over ordered pairs of POIs, where a pair (a, b) counts only if a comes before b in *both* the prediction and the truth.



ordered pairs (i before j):

Y: (A,B) (A,C) (A,D) (B,C) (B,D) (C,D)

$\hat{Y}$: (A,C) (A,B) (A,D) (C,B) (C,D) (B,D)

shared, SAME order: (A,B) (A,C) (A,D) (C,D) (B,D) → 5 of 6

(B,C) is in Y but $\hat{Y}$ has (C,B) — wrong order, so it does NOT count.

$$\text{precision} = 5/6, \text{ recall} = 5/6 \rightarrow \text{pairs-F1} = 2 \cdot P \cdot R / (P + R) = 5/6 \approx 0.833$$

Figure 4 — A worked pairs-F1: swapping two middle POIs breaks one ordered pair → 5 of 6 survive → 0.83. (In this task the route length K is fixed, so precision = recall = F1.)

3. Phase 1 — the personalized, context-aware engine

The engine is a **next-POI model** that fuses three signals into a score over all POIs: a **graph** of POIs (geographic kNN $\cup$ co-visit) encoded by a 2-layer **GCN**; per-step **context** ($\Delta d, \Delta t$) encoded by small MLPs; and a **user embedding**. A GRU reads the sequence so far, and an MLP head scores the next POI. The user embedding is the title's *user preferences*; the $\Delta d/\Delta t$ features are the *contextual data*.

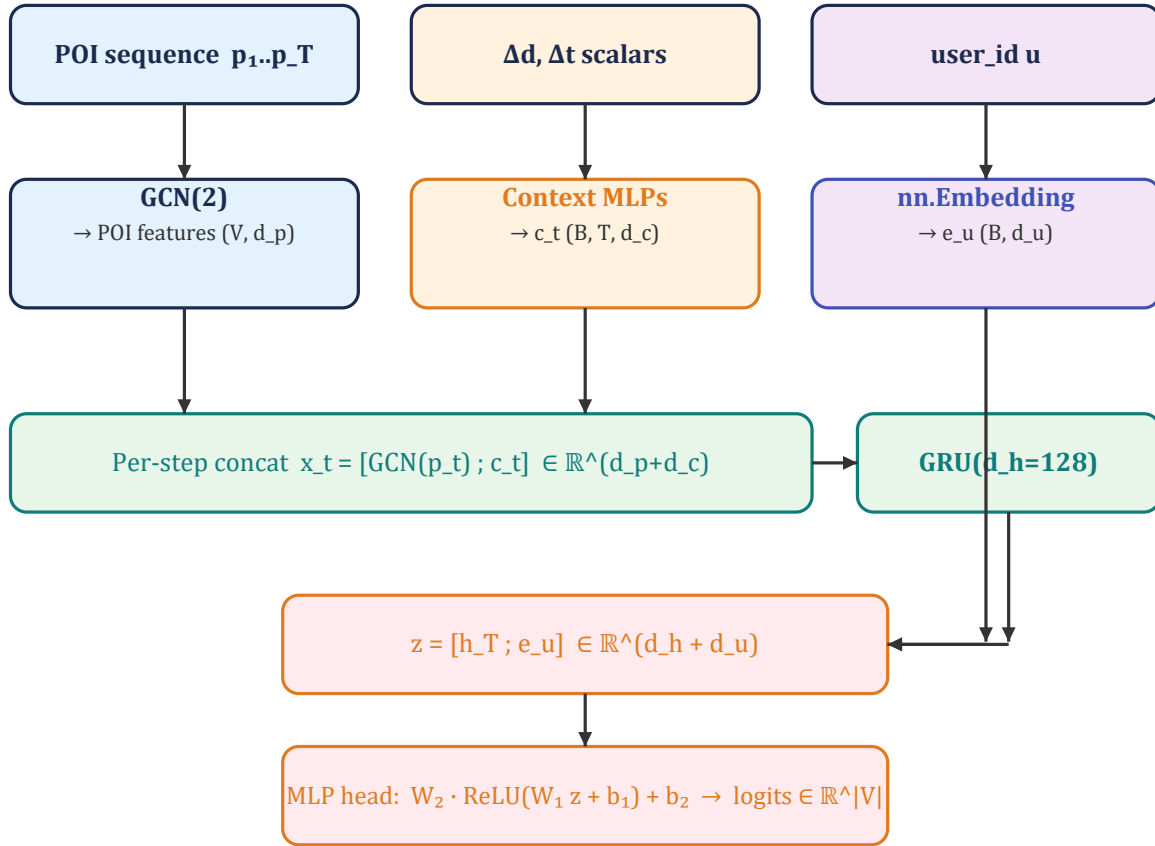


Figure 5 — The next-POI engine (Phase 1): graph + context + user $\rightarrow$ GRU $\rightarrow$ score over all POIs. This single model is reused by every itinerary strategy.

Phase-1 result. On Foursquare NYC (full vocabulary) the engine reaches **HR@1 = 0.187** (HR@10 = 0.588, MRR = 0.316) — the LSTM/STGCN tier of the LLM4POI benchmark. An honest, leakage-free baseline, below the transformer/LLM state of the art but a strong engine.

Model	HR@1
LSTM	0.130
STGCN	0.180
Ours (GCN+GRU+user)	0.187
STAN	0.220
GETNext	0.240
STHGCN	0.270
LLM4POI	0.340

Table 1 — HR@1 tier (LLM4POI table). Ours sits between STGCN and STAN.

4. From engine to itinerary — two strategies

There are two ways to turn the engine into a full itinerary — and they are exactly the **decoupled** vs **integrated** options the field debates. **Strategy A** decodes the *frozen* engine: start at the start POI, repeatedly append the best next POI (masking visited ones, reserving the end), with no extra training. **Strategy B** trains a *dedicated* pointer model end-to-end on whole trajectories.

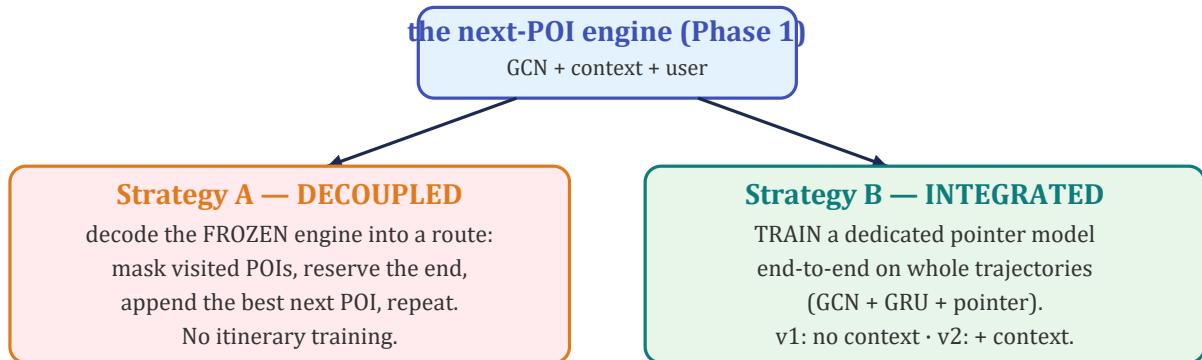


Figure 6 — One engine, two strategies. A = decoupled decoding (the “separate problems” approach); B = an integrated end-to-end itinerary model.

5. The integration experiment — A vs B (NYC)

Comparing A and B directly **tests the field’s integration hypothesis**. The result is honest and instructive — on Foursquare NYC (length ≥ 3 test trajectories):

Method	pairs-F1	kind
A — frozen rollout (greedy)	0.289	decoupled
A — frozen rollout (beam 3)	0.290	decoupled
B-v1 — pointer (no context)	0.259	integrated
B-v2 — pointer (+ context)	0.261	integrated

Table 2 — Strategy A (decoupled) vs B (integrated), pairs-F1.

The integrated model did NOT beat the decoupled one. The trained pointer (B) trailed the frozen rollout (A) by ~ 0.03 pairs-F1. Why: the next-POI engine was trained on $\sim 75,000$ prefix $\rightarrow$ next examples, whereas the integrated pointer saw only $\sim 10,000$ whole trajectories. **Lesson: integration does not automatically win — supervision density matters.** This *nuances* Halder/DLIR rather than contradicting it.

6. The comparability problem, and Strategy D

Our NYC itinerary pairs-F1 (~ 0.29) looks far below the literature’s ~ 0.6 – 0.8 — but that is a **comparability gap**, not a quality gap. Two scores compare only if the **dataset**, **protocol**, and **metric** all match. The literature uses the small **Flickr** photo-trajectory datasets (27–88 POIs) with leave-one-trajectory-out CV; NYC has $\sim 5,000$ POIs and a fixed split. So **Strategy D** re-runs the itinerary

task on the Flickr datasets, under the exact published protocol, using Chen 2016's own trajectory files (so our trips are *identical* to the literature's).

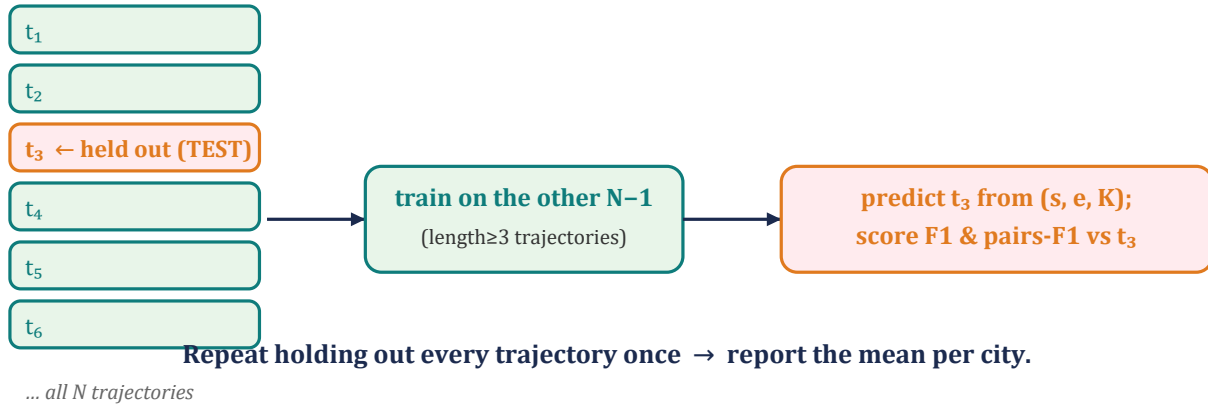


Figure 7 — Leave-one-trajectory-out CV: hold out one trip, train on the rest, predict it, repeat for all, average. The protocol the Flickr papers use.

City	#POIs	#users	#trips	#trips ≥ 3 (evaluated)
Toronto	29	1,395	6,057	335
Osaka	27	450	1,115	47
Glasgow	27	601	2,227	112
Edinburgh	28	1,454	5,028	634
Melbourne	88	1,000	5,106	442

Table 3 — The five Flickr cities. The $\text{length} \geq 3$ column is the evaluated set.

7. Validation — we reproduce Chen 2016

The decisive fairness check: a **Random** baseline has no modelling choices, so it can only match Chen 2016's Random if our data, protocol, and metric are identical to the paper's. They match within noise on every city — proving the comparison is valid.

pairs-F1	Toronto	Osaka	Glasgow	Edinburgh	Melbourne
Random — ours	0.298	0.301	0.301	0.270	0.227
Random — Chen 2016	0.310	0.304	0.320	0.261	0.248
PoiPopularity — ours	0.443	0.413	0.510	0.439	0.320
PoiPopularity — Chen 2016	0.384	0.365	0.507	0.436	0.316

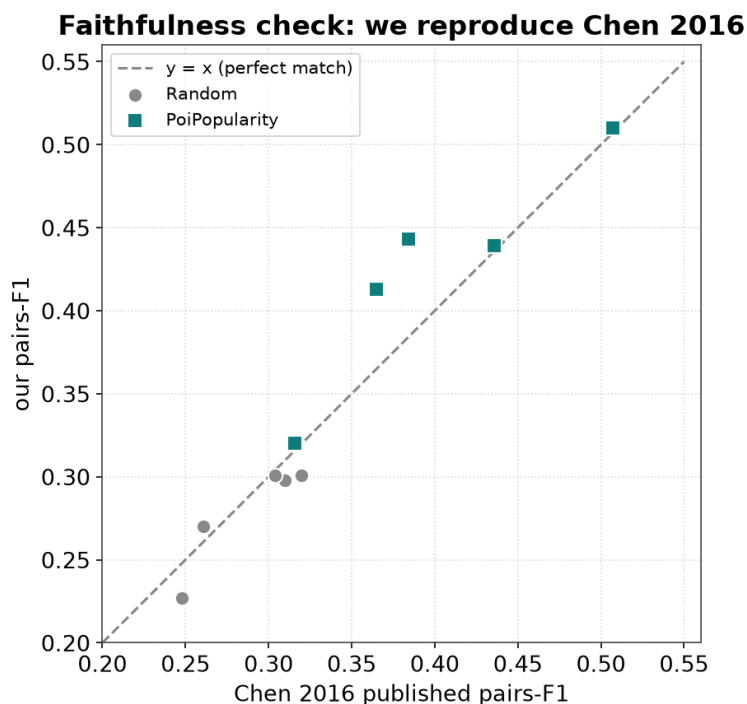


Figure 8 — Our pairs-F1 vs Chen 2016's; Random sits on the $y = x$ line.

8. Strategy D — results on the literature's scale

Our measured pairs-F1 (leave-one-out, length ≥ 3). The classical Markov baseline is the strongest of ours; the learned GCN+pointer reaches the published scale but trails Markov and the neural SOTA — the same lesson as the NYC integration experiment.

pairs-F1	Toronto	Osaka	Glasgow	Edinburgh	Melbourne
Random	0.298	0.301	0.301	0.270	0.227
PoiPopularity	0.443	0.413	0.510	0.439	0.320
Markov	0.504	0.421	0.587	0.449	0.333
MarkovPath	0.528	0.398	0.543	0.452	0.346
Pointer (learned)	0.431	0.399	0.489	0.414	0.312

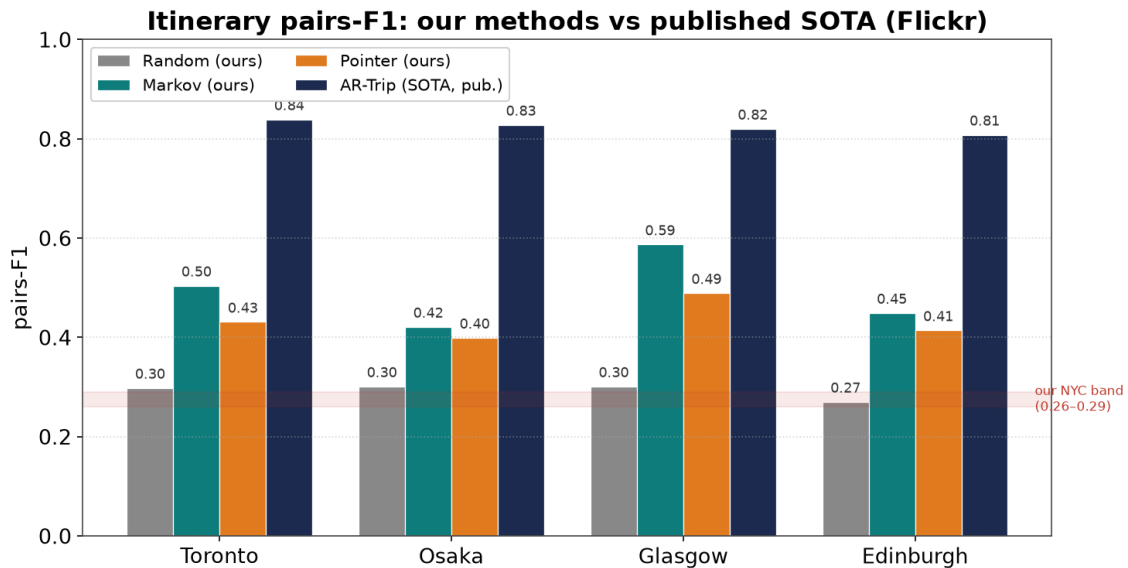


Figure 9 — Our methods beside the published SOTA (AR-Trip). Everything is on the literature's 0.3–0.85 scale; the red band is where our NYC numbers sat.

pairs-F1 (published)	Toronto	Osaka	Glasgow	Edinburgh
PoiRank (Chen)	0.518	0.511	0.548	0.432
Rank+Markov (Chen)	0.512	0.486	0.545	0.444
DeepTrip	0.748	0.755	0.782	0.660
CTLTR	0.748	0.719	0.763	0.681
SelfTrip	0.835	0.851	0.818	0.779
AR-Trip	0.839	0.828	0.820	0.808

Table 4 — Published pairs-F1 (comparable subset). No single SOTA across cities: AR-Trip leads on 3 cities, SelfTrip on Osaka.

9. Positioning vs Halder / DLIR

Halder's 2024 survey defines the personalized-itinerary field; **DLIR (2025)** argues that treating POI recommendation and itinerary construction as *separate* problems is sub-optimal, and proposes an **integrated** model that also adds **dynamic temporal interest**, **queuing time**, and **time-budget scheduling**.

- **We test Halder's central claim directly** (Strategy A vs B) and report a nuanced result: integration did not automatically win on our data.
- **We model** user preference (embedding) and spatial-temporal context ($\Delta d, \Delta t$) — the title's two pillars.
- **We deliberately scope smaller** than DLIR: we do not yet model queuing, explicit time budgets, or time-of-day dynamics. These are the path to a full module (Section 11).

10. Honest limitations

- **Integration didn't beat decoupling on NYC** — a supervision-density effect, not a refutation of the idea.

- **Personalization is uneven** — strong on Foursquare (user embedding), light in the comparable Flickr results.
- **Context is light** — $\Delta d/\Delta t$ only; no time-of-day dynamics, queuing, or budgets.
- **Markov is a hard baseline** — on tiny data, a simple transition model beats the neural pointer.
- **Single seed; NYC only for Phase 1** — multi-seed and TKY runs are planned.

11. Future work — toward the full Smart Visit Module

- **Integrate properly** — warm-start / share the engine with the pointer; emit prefix sub-trajectories to match supervision density.
- **Personalize the itinerary** — turn on the pointer's user-embedding lever; report personalized-vs-not pairs-F1 on Flickr.
- **Richer context** — time-of-day / day-of-week dynamic interest, opening hours.
- **Add scheduling** — explicit time budget + queuing time (the DLIR direction) for realistic day plans.
- **Robustness** — multi-seed, more cities, component ablations.

12. In one sentence

A personalized, context-aware next-POI engine, turned into itineraries two ways (decoupled vs integrated) and honestly benchmarked against the literature on the standard Flickr datasets — testing, and nuancing, the field's integration hypothesis, with a clear path toward a full scheduling-aware Smart Visit Module.